CHITTAGONG UNIVERSITY OF ENGINEERING & TECHNOLOGY

DEPARTMENT OF ELECTRONICS AND TELECOMMUNICATION ENGINEERING

PROJECT REPORT

NAME OF THE EXPERIMENT:

Smart Industrial Environment Monitoring & Control System

COURSE NO.	: ETE 416
COURSE TITLE	: IoT and Industrial Automation Sessional
LEVEL	: 4
TERM	: II
Group	: 3
DATE OF SUBMISSION	: 05.04.2026

Submitted To :

Taieba Taher
Assistant Professor
ETE, CUET

Submitted By :

Name : Fariha Tasnim Arpa
ID : 2008006
Name : Turja Talukder
ID : 2008022
Name : Nusrat Jahan
ID : 2008056

Introduction

Modern industries require continuous monitoring of environmental parameters such as temperature, humidity, gas concentration, and distance. Delayed detection of hazardous conditions such as gas leakage or high temperature can bring upon disastraus consequences. Traditional manual monitoring systems are inefficient and prone to human error as well as costly and not scalable. It is also hard to monitor real-time data and control remotely.

The advancement of Internet of Things (IoT), enabled real-time monitoring and automated control. A centralized monitoring system can be developed with the help of IoT system ensuring safety and efficiency. This project presents an IoT-based industrial monitoring and control system using ESP32, multiple sensors, MQTT communication, and Node-RED dashboard for visualization and control. The designed system provides continuous monitoring of environmental parameters such as temperature, humidity, gas concentration and distance. Sensor data are transmitted in real time using MQTT protocol to a Node-RED based dashboard.

Motivation

The motivation of this project is driven by the need for:

- Early detection of hazardous conditions
- Real-time monitoring of industrial environments
- Remote monitoring through cloud system flowfuse
- Automated control systems to reduce human intervention
- Improved safety and operational efficiency

Objectives

- To design an IoT-based monitoring system using ESP32
- To measure temperature, humidity, gas concentration and distance
- To implement MQTT-based communication
- To develop a real-time dashboard using Node-RED
- To implement automation logic for industrial safety
- To display real-time data on LCD and dashboard

Block Diagram

The system consists of ESP32 microcontroller connected with multiple sensors and actuators. Sensor data are transmitted via WiFi to Node-RED dashboard using MQTT protocol. The system also includes LCD display for local monitoring and relay-based actuator control.

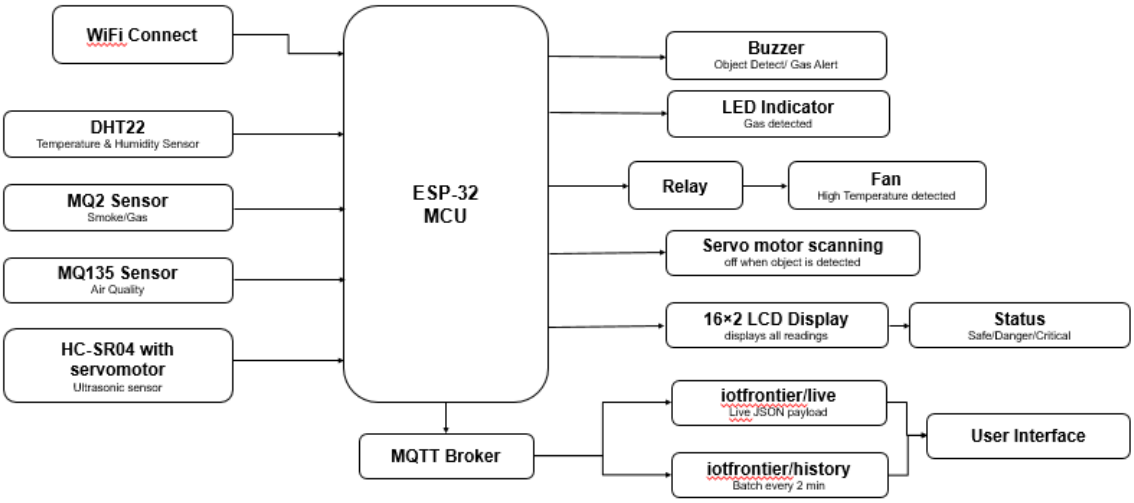


Figure 1: Block Diagram of the System

Hardware Components

- ESP32 Microcontroller
- DHT22 Temperature & Humidity Sensor
- MQ2 Gas Sensor
- MQ135 Air Quality Sensor
- HC-SR04 Ultrasonic Sensor
- Relay Module (Active LOW)
- Buzzer
- Servo Motor
- 16x2 LCD Display (I2C)

Software Elements

- MQTT Server
- Flowfuse

System Architecture

The system follows a layered IoT architecture:

- **Perception Layer:** DHT22, MQ2, MQ135, HC-SR04 sensors
- **Network Layer:** WiFi communication using ESP32
- **Communication Protocol:** MQTT and HTTP
- **Middleware Layer:** Node-RED server
- **Application Layer:** Dashboard and User Interface visualization
- **Control Layer:** Relay, Buzzer,LED, Fan, Servo motor

Workflow Description

Sensors collect environmental data. ESP32 processes the sensor readings and data are transmitted via MQTT to Node-RED. User panel and Dashboard displays real-time values and history. System compares values with thresholds. Actuators are triggered accordingly. Actuators can be triggered from the user interface as well.

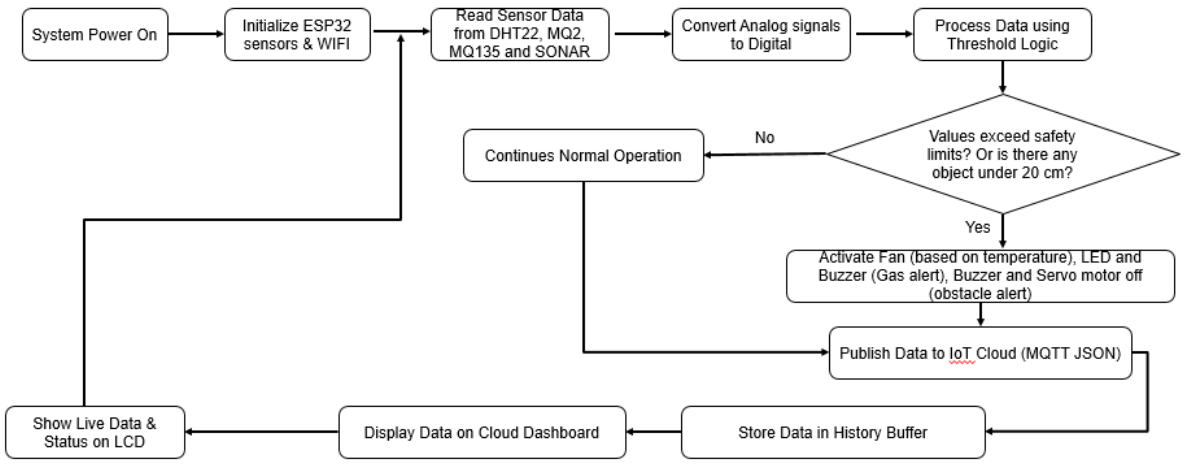


Figure 2: Working Flow of the System

System Features

- Real-time monitoring of temperature and humidity
- Gas detection using MQ2 and MQ135 sensors
- Distance monitoring using ultrasonic sensor
- 180-degree scanning using servo motor
- Automatic fan control using relay

- Buzzer alert for dangerous distance and gas alert
- LCD-based local display
- Node-RED dashboard visualization
- Historical data logging and visualization

Circuit Diagram

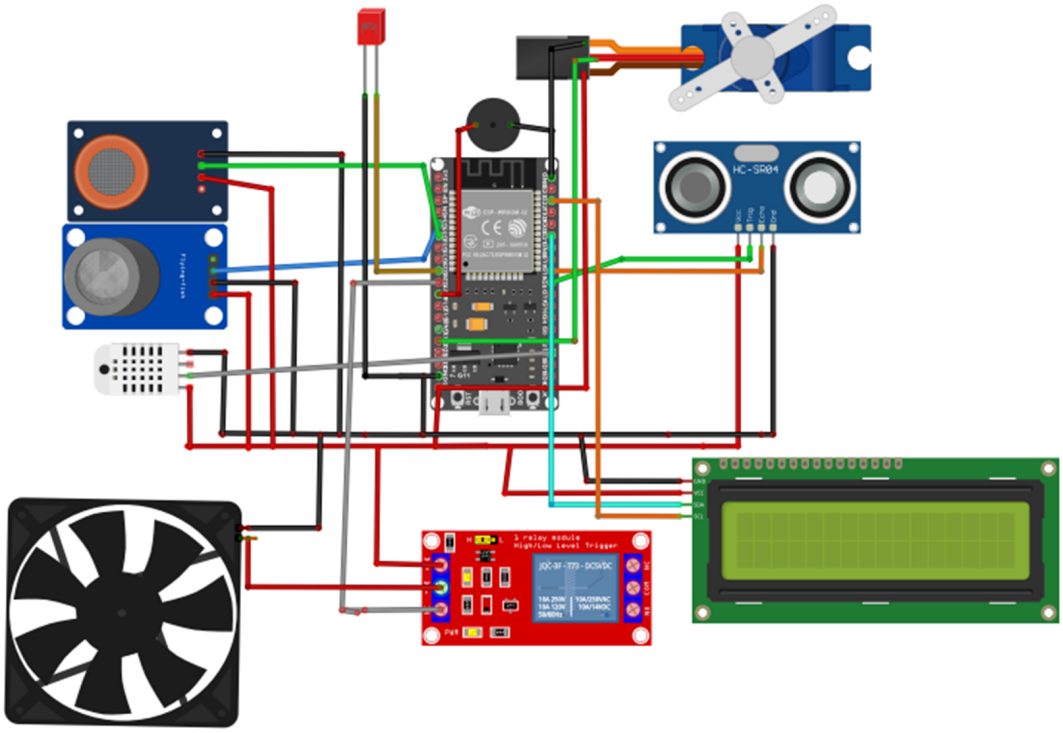


Figure 3: Circuit Diagram

Working Principle

The ESP32 collects sensor data at regular intervals. Temperature and humidity are measured using DHT22, gas concentration using MQ sensors and distance using ultrasonic sensor.

1. If temperature exceeds 40°C, the relay turns OFF (fan turns ON). Since the relay module operates in active LOW mode, a LOW signal from the ESP32 turns the relay ON, while a HIGH signal turns it OFF. System status is determined as SAFE or CRITICAL.
2. The servo motor scans across 180 degrees to detect objects in a wider range. Based on the collected data. If distance is less than 20 cm, buzzer is activated. System status is determined as SAFE or DANGER.

3. Different buzzer patterns were used for different conditions. When an object is detected within 20 cm, the buzzer produces a continuous alert. When gas concentration exceeds threshold, the buzzer produces a distinct alert pattern to differentiate between hazards.

4. If the air quality and gas detection exceeds their threshold values, the LED turns on upon detecting high gas level.

5. Manual and automatic control mechanisms are implemented for fan, LED and buzzer. When automatic conditions are triggered, the corresponding control switches on the dashboard are also updated automatically. Users can override the system using manual controls from the dashboard.

Sensor data is transmitted via MQTT using topics:

- `iotfrontier/live`
- `iotfrontier/history`

Additional MQTT topics are used for actuator control, including fan/control, led/control, and buzzer/control, enabling real-time remote operation.

Node-RED Implementation

Node-RED is used as the central processing and visualization unit.

Data Processing

Incoming MQTT messages are parsed and processed using function nodes. The payload is converted from JSON and separated into individual parameters.

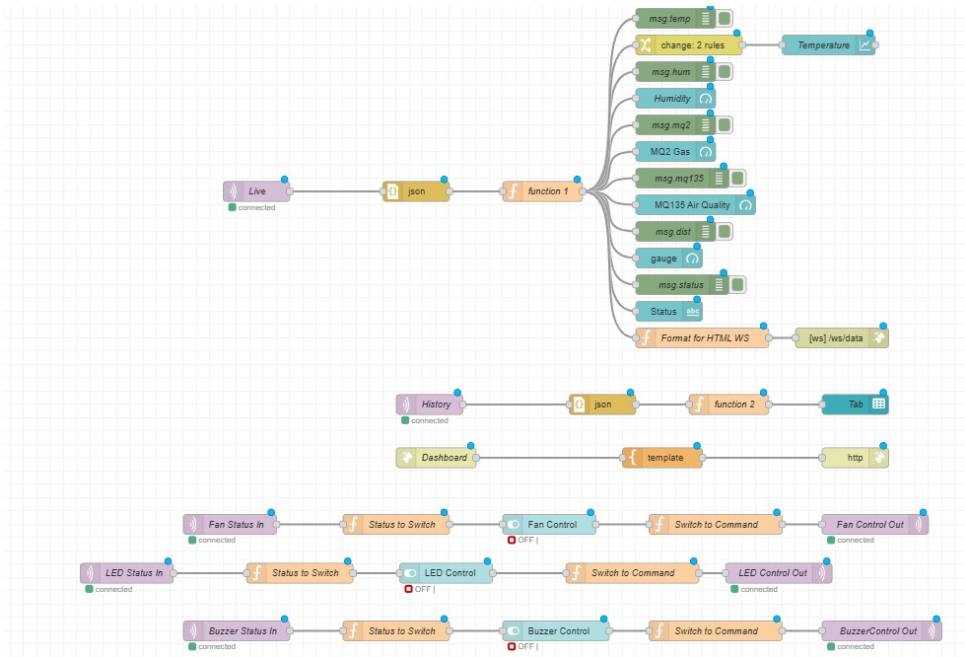


Figure 4: Node-red Interface

Dashboard

The dashboard displays:

- Temperature and humidity gauges
- Gas sensor readings
- Distance measurement
- System status (SAFE/DANGER)
- Historical data table

Control Strategy

- Temperature is continuously monitored and compared with threshold (40°C)
- When temperature exceeds threshold, relay is turned OFF to activate fan.Distance is monitored using ultrasonic sensor.When distance falls below 20 cm, buzzer is activated.
- System status is classified as SAFE or DANGER based on sensor values
- Servo motor provides 180-degree scanning coverage for detection
- Separate Switches are used to provide control for each of the fan,LED and buzzer.
- Both automatic and manual control mechanisms are implemented.
- When a threshold condition is triggered (e.g., temperature > 40°C), the system automatically activates the corresponding actuator and updates the dashboard control switch.
- Users can manually override the system using dashboard switches for fan, LED, and buzzer.
- MQTT is used for bidirectional communication between ESP32 and dashboard.
- Threshold values are experimentally set as MQ2 ≥ 300, MQ135 ≥ 3500 and distance threshold $= 20$ cm.

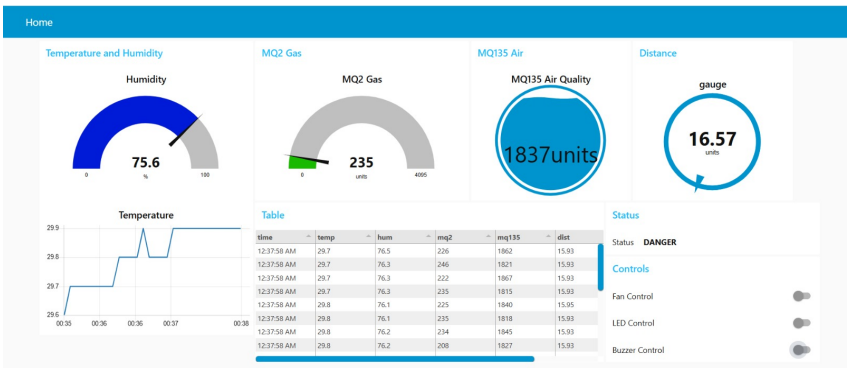
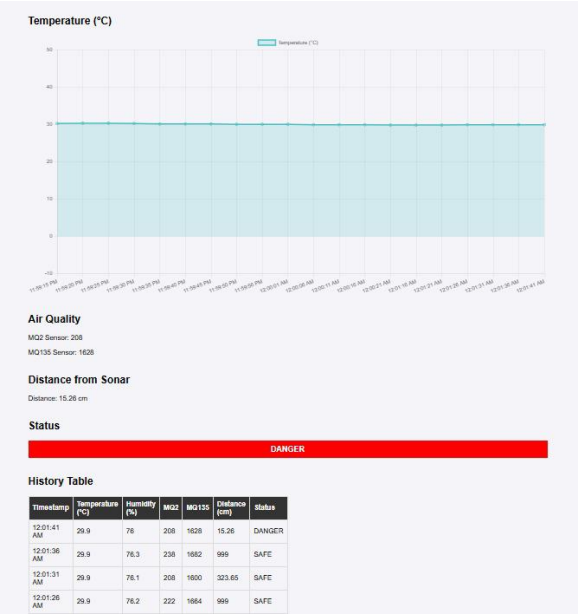


Figure 5: Admin Interface for Remote Monitoring and Control

Web Dashboard

A custom HTML dashboard is implemented using WebSocket communication. Real-time updates are visualized using script.js.



(a) During Danger Status

(b) During Safe Status

Figure 6: Node-RED Dashboard View

LCD Display System

A 16x2 I2C LCD is used to display real-time sensor data. Multiple pages are implemented:

- Temperature and humidity
- Gas sensor readings
- Distance measurement
- System status

Automation Logic

- Temperature-based relay control (fan automation)
- Distance-based and gas alart-based buzzer activation
- System status is classified into SAFE, DANGER and CRITICAL states. CRITICAL condition occurs when multiple hazardous conditions (e.g., high temperature and object detection) occur simultaneously.

LED Indicator

An LED indicator is implemented for gas detection. When gas concentration from MQ2 or MQ135 sensors exceeds predefined threshold values, the LED is turned ON automatically. The LED can also be manually controlled from the dashboard using MQTT-based commands.

Smart Control Synchronization

The system implements synchronized control between automatic logic and user interface. When an automatic condition is triggered (e.g., high temperature), the corresponding dashboard switch is updated automatically. Similarly, manual control actions from the dashboard are transmitted to the ESP32 using MQTT.

This ensures real-time bidirectional communication and provides both automation and user control flexibility.

Security Considerations

- Secure Wi-Fi communication between ESP32 and server
- MQTT communication with controlled access
- API-based authentication for data transmission
- Restricted access to dashboard
- Basic protection against unauthorized data access

Results and Observations

- Sensor data were successfully acquired and processed in real time.
- The system was able to detect environmental changes accurately.
- Data were transmitted to Node-RED dashboard with minimal delay.
- Automation logic was executed correctly based on threshold conditions.
- The LCD display showed real-time updates effectively.
- The system successfully synchronized automatic control actions with dashboard interface in real time.
- The system demonstrated reliable real-time response with minimal latency between sensor detection and actuator activation.

Limitation

The 16×2 LCD display can show only limited data at a time, requiring a page-based update system. Due to the delay between page transitions, real-time monitoring of all parameters simultaneously is not possible and some data updates may be missed.

Future Works

- Mobile application development
- AI-based predictive monitoring
- Enhanced security implementation
- Using flame sensor and OLED display

Conclusion

The project successfully demonstrated an IoT-based industrial monitoring and control system. It integrated sensors, embedded systems, and cloud-based visualization to provide real-time monitoring and automation. Minor fluctuations were observed in gas sensor readings. Network latency has slightly affected real-time visualization. Servo-based scanning improved coverage area for detection. The system reduced the need for manual monitoring.

ESP32 Code

`https://github.com/Turja14/Smart-Industrial-Environment-Monitoring-Control-System`